Matplotlib & Pyplot Interview Code Reference

Core Plotting Workflows, Subplot Architectures, & Interview Coding Traps

1. Object-Oriented (OO) vs Pyplot State-Machine Interface

Interviews strongly test your understanding of **Object-Oriented (OO) interface** versus implicit pyplot state-machine syntax. Always prefer the explicit OO paradigm (fig, ax = plt.subplots()) for clarity and multi-plot control.

```
import matplotlib.pyplot as plt
import numpy as np

# Recommended OO Paradigm: Explicit Figure & Axes Objects
fig, ax = plt.subplots(figsize=(7, 4), dpi=100)
x = np.linspace(0, 10, 100)
ax.plot(x, np.sin(x), color='#0284C7', linewidth=2, label='sin(x)')

# Explicit Axes Customization
ax.set_title("Sine Wave Function", fontsize=12, fontweight='bold')
ax.set_xlabel("Time (s)")
ax.set_ylabel("Amplitude")
ax.legend(loc='upper right')
ax.grid(True, linestyle='--', alpha=0.5)

# Save high-resolution output cleanly
plt.savefig('sine_wave.png', dpi=300, bbox_inches='tight')
plt.close(fig) # Always close figure in production to free memory!
```

2. Essential Plot Types & Exact Syntax

Plot Type	Key Syntax	Common Keyword Arguments
Line Plot	ax.plot(x, y)	color, linestyle ('--', '-.'), linewidth, marker ('o', 's')
Scatter Plot	ax.scatter(x, y)	c (array/colors), s (marker size), cmap, alpha
Bar Chart	ax.bar(x, height) ax.barh(y, width)	width, bottom (stacked bars), align='center'
Histogram	ax.hist(data)	bins, density=True, cumulative=True, edgecolor
Box Plot	ax.boxplot(data)	vert=True/False, patch_artist=True, showmeans=True
Heatmap / Image	ax.imshow(matrix)	cmap='coolwarm', interpolation='nearest', origin='lower'

3. Subplots & Advanced Multi-Panel Architectures

Handling subplots is a prime interview question. Master grid indexing, axes.flat, dual Y-axes (twinx()), and custom GridSpec layouts.

```
# Pattern A: Standard 2x2 Grid with Shared Axes
fig, axes = plt.subplots(2, 2, figsize=(8, 6), sharex=True, sharey=True)

# Iterate flat array of axes
for i, ax in enumerate(axes.flat):
    data = np.random.randn(100)
    ax.hist(data, bins=15, color='#0284C7', edgecolor='white', alpha=0.8)
    ax.set_title(f"Sample Group {i+1}", fontsize=9)

plt.tight_layout() # Prevents overlapping labels & titles!

# Pattern B: Dual Y-Axis (Twin Axes)
fig, ax1 = plt.subplots(figsize=(7, 3.5))
ax2 = ax1.twinx() # Creates a second Y-axis sharing the same X-axis

l1 = ax1.plot(x, np.exp(x/2), 'b-', label='Exponential Growth')
l2 = ax2.plot(x, np.sin(x), 'r--', label='Oscillation')

ax1.set_ylabel('Growth Rate (Exp)', color='b')
ax2.set_ylabel('Oscillation Amplitude', color='r')

# Combine legends from both axes cleanly
lines = l1 + l2
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper left')
```

4. Customization, Tick Formatting & Annotations

```
fig, ax = plt.subplots(figsize=(7, 3.8))

# Plot bar chart with custom styling
categories = ['Alpha', 'Beta', 'Gamma', 'Delta']
values = [45, 88, 62, 91]
bars = ax.bar(categories, values, color='#1E293B', width=0.5)

# Customizing Tick Marks and Labels
ax.set_ylim(0, 110)
ax.set_ylabel("Performance Metric (%)", fontweight='bold')
ax.set_title("Q4 System Performance Summary", fontweight='bold', pad=10)

# Removing Top and Right Spines for a clean modern design
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.grid(axis='y', linestyle=':', alpha=0.7)

# Adding Direct Annotations / Value Labels on Top of Bars
for bar in bars:
    height = bar.get_height()
    ax.annotate(f'{height}%',
                xy=(bar.get_x() + bar.get_width() / 2, height),
                xytext=(0, 3), # 3pt vertical offset
                textcoords="offset points",
                ha='center', va='bottom', fontweight='bold', fontsize=9)
```

5. Critical Coding Interview Traps & Pitfalls

Interview Trap / Topic	The Mistake	Correct Production Solution
Memory Leaks in Loops	Calling plt.plot() inside a long loop without closing figure.	Always call plt.close(fig) or plt.close('all') inside iteration loops.
plt.cla() vs plt.clf() vs plt.close()	Confusing clearing axes vs figure vs closing window handle.	cla() = clears current Axes. clf() = clears current Figure. close() = closes window & frees memory.
Overlapping Subplot Labels	X-axis labels or titles overlapping adjacent subplot ticks.	Call plt.tight_layout() or fig.constrained_layout=True before rendering.
Modifying Data Views vs Copies	Passing NumPy array slice to plot, then modifying array in-place.	Use arr.copy() if underlying array will undergo mutation.
Stateful vs OO Subplot Confusion	Mixing plt.subplot(2,1,1) with ax.plot() calls unexpectedly.	Stick 100% to OO style: fig, (ax1, ax2) = plt.subplots(2, 1).

PRO TIP FOR CODING INTERVIEWS

When asked to plot data in Python data science interviews, start by creating a clean figure/axes pair using `fig, ax = plt.subplots()`. Always include titles, axis labels, legend (if >1 series), and call `plt.tight_layout()`. Mention memory management (`plt.close(fig)`) when writing code for production pipelines.